

Opcode Extraction from Malicious Executables Using Open-Source Reverse Engineering Tools

CIS*6530 Cyber Threat Intelligence and Adversarial Risk Analysis

Prepared By:

Khushbukumari Hemalkumar Patel
Suprim Devkota

CONTENTS

INTRODUCTION	2
RATIONALE FOR SELECTING GHIDRA	2
ENVIRONMENT SETUP	3
RUNNING THE SCRIPT	3
SYSTEM DESIGN / METHODOLOGY	4
RESULTS	6
CONSIDERATION / LIMITATION	7
FUTURE STUDIES/DISCUSSIONS	8
REFERENCES	9

INTRODUCTION

In the previous submission, a collection of malicious executables and payloads associated with various Advanced Persistent Threat (APT) groups was collected. Building upon that dataset, this submission focuses on extracting operation codes (Opcodes) from those executables to enable structured static analysis and feature-level examination.

Opcode extraction was performed using Ghidra, an open-source software reverse engineering framework developed by the National Security Agency (NSA). To ensure scalability and reproducibility, the extraction process was fully automated using a headless analysis workflow. A Python-based script was developed that orchestrates large-scale parallel processing for programmatically extracting Opcodes from disassembled binaries.

This report documents the complete workflow, including environment configuration, architectural design, implementation details, execution methodology, and validation mechanisms. The proposed framework ensures reproducibility, structured logging, safe failure handling, and consistent opcode extraction across multiple APT-associated malware executables.

RATIONALE FOR SELECTING GHIDRA

Several reverse engineering platforms are widely used for static binary analysis, including IDA Pro, Binary Ninja, and Ghidra.

IDA Pro is considered an industry-standard reverse engineering tool, offering extensive architecture support, powerful debugging capabilities, and a mature plugin ecosystem. However, it is a commercial product with significant licensing costs, and its free version provides limited functionality. [1]

Binary Ninja offers an intuitive interface and strong interactive analysis capabilities, along with a flexible scripting API. Nevertheless, its full feature set is also restricted to licensed versions, and its architecture support is narrower compared to IDA Pro and Ghidra. [2]

In contrast, Ghidra provides the following advantages:

- Free and open-source licensing model
- Extensive architecture and binary format support (PE, ELF, Mach-O, XCOFF, etc.)
- Built-in disassembler and decompiler
- Headless automation support for batch processing
- Python-based scripting framework for extensibility
- Active community support and continuous updates

The availability of a headless analysis mode was particularly critical for this submission, as it enabled automated large-scale opcode extraction without relying on graphical interaction. Additionally, its open-source nature ensures transparency, reproducibility, and accessibility, making it well-suited for academic research and malware analysis workflows. [2] [1]

ENVIRONMENT SETUP

The scripts are executed within a Linux environment. As the opcode extraction is performed using Ghidra, The setup steps are as follows:

1. **Download Ghidra:** Obtain the required version from the official repository.
<https://github.com/NationalSecurityAgency/ghidra>

2. **Extract and Clean Up:** Unzip the downloaded folder and remove the original archive:

```
unzip ghidra_12.0.3_PUBLIC_20260210.zip  
rm ghidra_12.0.3_PUBLIC_20260210.zip
```

3. **Install Java Development Kit (JDK):** If not already installed, download and install JDK 21, which is required for this submission: <https://adoptium.net/temurin/releases>.

Extract and remove the archive:

```
tar -xf OpenJDK21U-jdk_x64_linux_hotspot_21.0.10_7.tar.gz  
rm OpenJDK21U-jdk_x64_linux_hotspot_21.0.10_7.tar.gz
```

4. **Python Requirement:** The automation manager script requires Python 3. It is assumed that Python 3 is pre-installed in the Linux environment.

This environment setup ensures that both Ghidra and the required Java runtime are properly configured to execute the analysis scripts efficiently.

RUNNING THE SCRIPT

Running the scripts can be done by following the command:

```
python3 opcode_extraction_pipeline.py \  
--ghidra "<Path_To_Ghidra_Headless_Analyzer>" \  
--samples "<Path_To_The_Malware_Sample_Directory>" \  
--opcodes "<Path_To_Opcde_Output_Directory>" \  
--instructions "<Path_To_Instruction_Output_Directory>" \  
--threads "<Number_Of_Threads>"
```

As an example, the script can be run in the following manner:

```
python3 opcode_extraction_pipeline.py \  
--ghidra "/opt/ghidra/ghidra_12.0.3_PUBLIC/support/analyzeHeadless" \  
--samples "/home/CIS6530/Submission 2/Executable Malware/G0033_PoseidonGroup" \  
--opcodes "/home/CIS 6530/Submission 3/output/opcodes" \  
--instructions "/home/CIS 6530/Submission 3/output/instructions" \  
--threads 1
```

SYSTEM DESIGN / METHODOLOGY

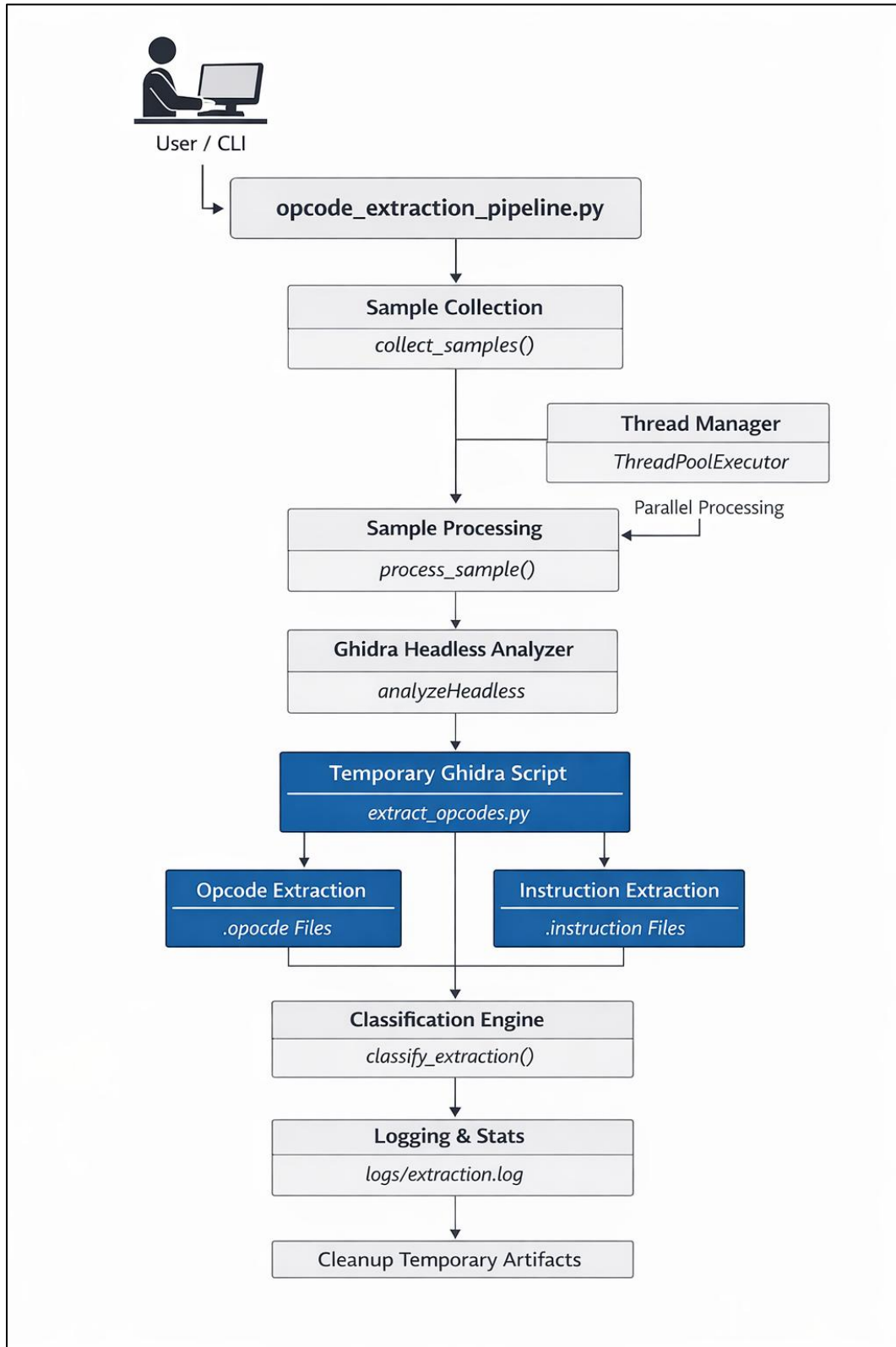


Figure 1: System Design for Opcode extraction pipeline

The **Opcode Extraction Pipeline** is designed to efficiently process malware samples and extract opcode and assembly instruction sequences in a scalable, automated manner. The architecture is composed of several modular components, as illustrated in the system diagram above. All these components work together to convert a batch of executables into the required Opcodes. The major steps are outlined below:

1. **Sample Collection** (*collect_samples*): A user uses a terminal to run the script above. This causes the script to recursively collect the malware samples from the user-specified input directory. Each sample is passed to the processing pipeline, regardless of file type, allowing Ghidra to determine format compatibility internally.
2. **Thread Manager** (*ThreadPoolExecutor*): Once the samples have been collected, they are parallelly submitted to a pipeline that supports multi-threaded execution. Each malware sample is assigned to a separate worker thread to enable parallel processing, improving throughput for large datasets.
3. **Sample Processing** (*process_sample*): Each malware sample is processed in isolation within a temporary Ghidra project to avoid conflicts. Validation is performed to skip zero-byte or unsupported files.
4. **Ghidra Headless Analyzer** (*analyzeHeadless*): The malware sample is imported into Ghidra's headless analysis environment. Automatic disassembly and program analysis are performed without requiring a GUI, ensuring reproducibility and scalability.
5. **Temporary Ghidra Script** (*extract_opcodes.py*): A dynamically generated Jython script is executed within Ghidra to perform opcode and full instruction extraction. It writes *.opcode* files containing mnemonic sequences and *.instruction* files containing full assembly instructions.
6. **Classification Engine** (*classify_extraction*): After extraction, each sample is classified based on instruction counts as a successful extraction, packed or obfuscated binaries, or failed extraction due to format or analysis errors.
7. **Logging**: While the script is running, logs are generated dynamically that stores results at various stages of the execution. This helps in finding anomalies and tracking errors.
8. **Cleanup Temporary Artifacts**: All temporary Ghidra scripts and project directories are removed after analysis, ensuring that the workspace remains clean and ready for subsequent runs.

RESULTS

The script was run on a Kali Virtual machine from the terminal along with the appropriate arguments as shown below:

```
(kali@kali)~$ python3 opcode_extraction_pipeline.py --ghidra "/opt/ghidra/ghidra_11.3.2_PUBLIC/support/analyzeHeadless" --samples "Desktop/CIS6530/Submission 2/Executable Malware/G0089_The White Company" --opcodes "Desktop/CIS6530/Submission 3/output/opcodes" --instructions "Desktop/CIS6530/Submission 3/output/instructions" --threads 4
[+] Processing APT Group: G0089_The White Company
[+] Found 2 samples.
[+] Extraction completed.
[+] Temporary scripts cleaned.
[+] All temporary Ghidra projects cleaned.
[+] Summary written to logs/extraction.log
[+] Done.
```

Figure 2: Working example of script

The script then utilizes batch and parallel processing techniques to efficiently extract and output both opcodes and instructions to the specified directory. By processing multiple files simultaneously, the script significantly reduces execution time and improves overall performance, especially when handling large datasets. The resulting outputs are systematically organized in the designated directory, ensuring that the extracted opcodes and instructions are readily available for further analysis and preprocessing.

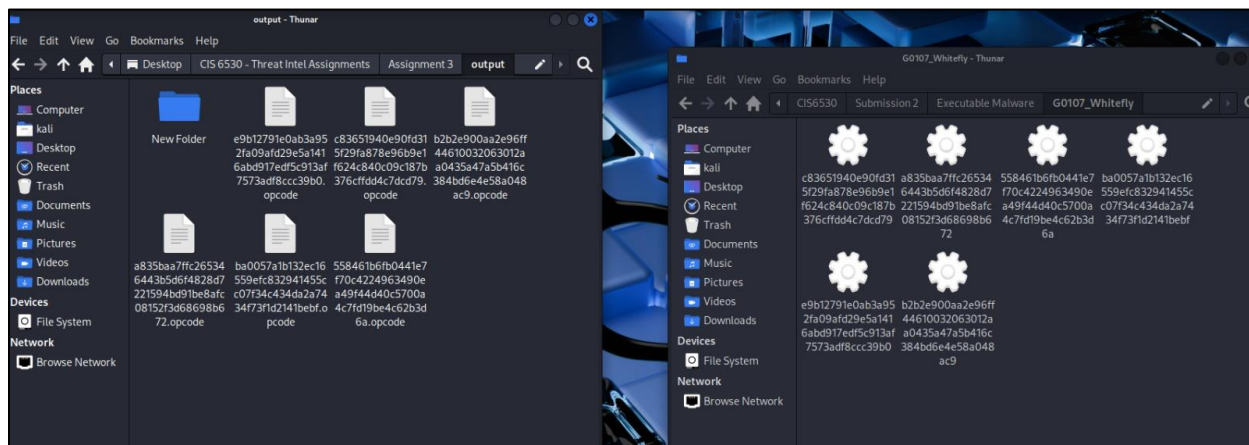


Figure 3: Opcode files generated by the script

Logging was enabled during the execution of the script to record the results generated for each individual executable. This logging mechanism allows the script to track the processing status, capture relevant outputs, and document any issues encountered during execution. By maintaining detailed logs for each executable, the process becomes more transparent, making it easier to review results, debug potential errors, and verify that the script operated correctly across all processed files.

```

2026-03-09 02:12:44,802 - INFO - SUCCESS (instruction count=38432): Desktop/CIS6530/Submission 2/Executable Malware/G0081_Tropic Troope
23dea3a74e3ff6a367754d02466db4c86ffda47efe09529d3aad52b0d5694b30
2026-03-09 02:15:26,353 - WARNING - POSSIBLE OBFUSCATED (very high instruction count=1372302): Desktop/CIS6530/Submission 2/Executable
Malware/G0081_Tropic Trooper/ea2f88847ee1b5a10a0286c5actb283a60b97b0a3325508b38900b16255e5589
2026-03-09 02:15:26,361 - INFO - =====
2026-03-09 02:15:26,361 - INFO - APT GROUP SUMMARY: G0081_Tropic Trooper
2026-03-09 02:15:26,361 - INFO - =====
2026-03-09 02:15:26,361 - INFO - Total : 6
2026-03-09 02:15:26,361 - INFO - Success : 5
2026-03-09 02:15:26,361 - INFO - Packed : 0
2026-03-09 02:15:26,361 - INFO - Obfuscated : 1
2026-03-09 02:15:26,361 - INFO - Failed : 0
2026-03-09 02:15:26,361 - INFO - Timeout : 0
2026-03-09 02:15:26,361 - INFO - Skipped : 0
2026-03-09 02:15:26,361 - INFO - =====
2026-03-09 02:15:44,903 - WARNING - SKIPPED OR UNSUPPORTED FORMAT Desktop/CIS6530/Submission 2/Executable Malware/G0050_APT32/
be2648d8ac9aca8c1fb338328dcd0a3bbb341e1b/qcblt/c/00ear809a1b47/
2026-03-09 02:15:57,284 - INFO - SUCCESS (instruction count=15936): Desktop/CIS6530/Submission 2/Executable Malware/
G0050_APT32/19f16a4eceb8b57b2bcad11c76446f05b1ef4b7c7f23201e08dc8fa07659pcf0
2026-03-09 02:16:00,699 - INFO - SUCCESS (instruction count=1404): Desktop/CIS6530/Submission 2/Executable Malware/
G0050_APT32/0241b90dff6b2c76bcae2c50ff1b4ald8957ffedd6b316ec9d4f0d454748959b
2026-03-09 02:16:00,701 - INFO - =====
2026-03-09 02:16:00,701 - INFO - APT GROUP SUMMARY: G0050_APT32

```

Figure 4: Logs generated by the script

CONSIDERATION / LIMITATION

- The automated script extracts nearly all assembly instructions from the binary, including compiler boilerplate and potentially unreachable code. In contrast, malware analysts rarely examine every instruction in a binary and instead rely on techniques such as searching for strings or reviewing exported and imported functions to quickly locate relevant sections of code.
- The script counts the total number of extracted instructions to identify potential edge cases, such as packed or obfuscated malware samples. However, using instruction count alone is not a reliable method for detecting packed binaries; tools such as PEiD are typically used for this purpose.
- The script may fail when processing corrupted or garbled executables. As an example, a file may begin with the “MZ” header, which identifies it as a Portable Executable (PE) file, but the remainder of the file may be corrupted or improperly structured. When this occurs, the script encounters errors during processing because the binary does not follow the expected PE format.
- The script skips .DMG malware because they are not supported by Ghidra’s Headless Analyzer. In contrast, .XCOFF malware is allowed because this type of malware can be directly disassembled by Ghidra.

FUTURE STUDIES/DISCUSSIONS

The results of this submission will ultimately be used to train a machine learning model designed to identify patterns in opcodes that may indicate Advanced Persistent Threat (APT) behavior. To ensure the effectiveness and reliability of the model, it is essential to first clean and pre-process the opcode instructions. This process involves removing noise, normalizing instruction formats, and structuring the data in a consistent way so that the model can accurately learn meaningful patterns. Proper preprocessing helps improve data quality, reduces ambiguity in the instruction set, and ensures that the resulting dataset is suitable for training a robust and reliable machine learning model.

REFERENCES

- [1] N. Zerof, "Ghidra vs IDA Pro: What the NSA's Free Reverse Engineering Toolkit Can Do - HackMag," HackMag, 12 September 2025. [Online]. Available: <https://hackmag.com/security/nsa-ghidra-2>.
- [2] "Binary Ninja vs. Ghidra: A Comprehensive Comparison for Reverse Engineering Enthusiasts," Oreate AI, 15 January 2026. [Online]. Available: <https://www.oreateai.com/blog/binary-ninja-vs-ghidra-a-comprehensive-comparison-for-reverse-engineering-enthusiasts/61cb0f38c41e32b5e0da3c022ebbe785>.
- [3] "NationalSecurityAgency/ghidra at Ghidra_12.0.3_build," National Security Agency, [Online]. Available: https://github.com/NationalSecurityAgency/ghidra/tree/Ghidra_12.0.3_build.
- [4] N. Fox, "How to Use Ghidra to Reverse Engineer Malware | Varonis," Varonis, 06 April 2023. [Online]. Available: <https://www.varonis.com/blog/how-to-use-ghidra>.
- [5] C. Kane, "CS6038/CS5138 Malware Analysis, UC by ckane," class.malware.re, 21 March 2021. [Online]. Available: <https://class.malware.re/2021/03/21/ghidra-scripting-feature-extraction.html>.
- [6] Libra, "Ghidra Tip 0x05: Headless execution - Max Kersten," maxkersten, 30 June 2021. [Online]. Available: <https://maxkersten.nl/2024/06/30/ghidra-tip-0x05-headless-execution/>.
- [7] "GitHub - HackOvert/GhidraSnippets: Python snippets for Ghidra's Program and Decompiler APIs," HackOvert, [Online]. Available: <https://github.com/HackOvert/GhidraSnippets?tab=readme-ov-file#count-all-mnemonics-in-a-binary>.